

19.04



1 perangkat

17,8 KB/dtk 4G



app.emergent.sh/



lanjutkan dengan konfigurasi default gabungan pilihan Anda:

Full dashboard (simulasi penuh + placeholder MQTT)

API + MongoDB untuk persistensi

Agent via Emergent Key (Claude Sonnet 4.5) untuk penalaran anomali

Auth

Personalisasi dark theme (gradien halus + mikro-animasi + tetap monospace terminal)

Integrasi playbook + design agent + eksplorasi struktur secara paralel:

History Fetched

History Fetched

original_problem_statement: "Simulasi uji kemampua..."

Design Agent messages

Showing 8 paths

index.tsx _layout.tsx .env package.json app.json

Frontend/app.json

```
{
  "expo": {
    "name": "frontend",
    "slug": "frontend",
    "version": "1.0.0",
    "orientation": "portrait",
    "icon": "./assets/images/icon.png",
    "scheme": "frontend",
    "userInterfaceStyle": "automatic",
    "newArchEnabled": true,
    "ios": {
      "supportsTablet": true,
      "bundleIdentifier": "com.emergent.neuronorchestrator.gq3ggy"
    },
    "android": {
      "adaptiveIcon": {
        "foregroundImage": "./assets/images/adaptive-icon.png",
        "backgroundColor": "#000000"
      },
      "edgeToEdgeEnabled": true,
      "package": "com.emergent.neuronorchestrator.gq3ggy"
    }
  },
  "name": "frontend",
  "version": "1.0.0",
  "description": "Frontend application for emergent integrations.",
  "main": "index.tsx",
  "scripts": {
    "start": "expo start",
    "android": "expo run:android",
    "ios": "expo run:ios"
  },
  "dependencies": {
    "react": "18.2.0",
    "react-native": "0.73.0",
    "expo-status-bar": "1.4.2",
    "expo-splash-screen": "0.27.3"
  },
  "devDependencies": {
    "@babel/core": "7.20.0",
    "@babel/preset-react": "7.18.6",
    "babel-plugin-transform-remove-console": "7.20.0",
    "expo-dev-client": "4.0.0",
    "expo-router": "3.5.1",
    "expo-updates": "0.21.0",
    "react-native-reanimated": "3.10.0",
    "react-native-safe-area-context": "4.5.0",
    "react-native-screens": "3.29.0",
    "react-native-web": "0.19.10"
  },
  "expo": {
    "output": {
      "path": "build"
    }
  }
}
```

Jul 16, 06:31 PM

ed /app/design_guidelines.json

Guidelines & integration playbook siap. Sekarang saya bangun backend & frontend paralel.

(server.py): FastAPI + MongoDB + emergentintegrations. Auto-seed 10 blocks.

19.04



1 perangkat

16,2 KB/dtk 4G



app.emergent.sh/



ik, saya lanjutkan dengan konfigurasi default gabungan pilihan Anda:

Hybrid dashboard (simulasi penuh + placeholder MQTT)

FastAPI + MongoDB untuk persistensi

LLM Agent via Emergent Key (Claude Sonnet 4.5) untuk penalaran anomali

Tanpa auth

Modernisasi dark theme (gradien halus + mikro-animasi + tetap monospace terminal)

ya jalankan integrasi playbook + design agent + eksplorasi struktur secara paralel:

✓ Memory Fetched

Memory Fetched

✓ original_problem_statement: "Simulasi uji kemampua..."

→ 3 Design Agent messages

Viewing 8 paths

.env

index.tsx

_layout.tsx

.env

package.json

app.json

/app/frontend/package.json

```
1 {
2   "name": "frontend",
3   "main": "expo-router/entry",
4   "version": "1.0.0",
5   "scripts": {
6     "preinstall": "./scripts/cmd-guard.js --preinstall",
7     "start": "expo start",
8     "reset-project": "node ./scripts/reset-project.js",
9     "android": "expo start --android",
10    "ios": "expo start --ios",
11    "web": "expo start --web",
12    "lint": "expo lint"
13  },
14  "dependencies": {
15    "@expo/metro-runtime": "6.1.2",
16    "@expo/vector-icons": "15.1.1",
17    "@react-native-async-storage/async-storage": "2.2.0",
18    "date-fns": "4.1.0",
19    "dayjs": "1.11.13",
20    "expo": "54.0.35",
21    "expo-blur": "15.0.8",
22    "expo-constants": "18.0.13",
```

Jul 16, 06:31 PM

Viewed /app/design_guidelines.json

design guidelines & integration playbook siap. Sekarang saya bangun backend & frontend paralel.

Plan>

ckend (server.py): FastAPI + MongoDB + emergentintegrations. Auto-seed 10 blocks.

19.04



1 perangkat

10,0 KB/dtk 4G



app.emergent.sh/



ik, saya lanjutkan dengan konfigurasi default gabungan pilihan Anda:

Hybrid dashboard (simulasi penuh + placeholder MQTT)

FastAPI + MongoDB untuk persistensi

LLM Agent via Emergent Key (Claude Sonnet 4.5) untuk penalaran anomali

Tanpa auth

Modernisasi dark theme (gradien halus + mikro-animasi + tetap monospace terminal)

ya jalankan integrasi playbook + design agent + eksplorasi struktur secara paralel:

✓ Memory Fetched

Memory Fetched

✓ original_problem_statement: "Simulasi uji kemampua..."

→ 3 Design Agent messages

Viewing 8 paths

.env

index.tsx

_layout.tsx

.env

package.json

app.json

/app/frontend/.env

```
1 EXPO_TUNNEL_SUBDOMAIN=neuron-orchestrator
2 EXPO_PACKAGER_HOSTNAME=https://neuron-orchestrator.preview.emergentagent.com
3 EXPO_PUBLIC_BACKEND_URL=https://neuron-orchestrator.preview.emergentagent.com
4 EXPO_USE_FAST_RESOLVER="1"
5 METRO_CACHE_ROOT=/app/frontend/.metro-cache
6 EXPO_PACKAGER_PROXY_URL=https://neuron-orchestrator.preview.emergentagent.com
7
8
```

Jul 16, 06:31 PM

Viewed /app/design_guidelines.json

sign guidelines & integration playbook siap. Sekarang saya bangun backend & frontend paralel.

Plan>

ckend (server.py): FastAPI + MongoDB + emergentintegrations. Auto-seed 10 blocks.



19.04



1 perangkat

8,00 KB/dtk 4G



app.emergent.sh/



ik, saya lanjutkan dengan konfigurasi default gabungan pilihan Anda:

Hybrid dashboard (simulasi penuh + placeholder MQTT)

FastAPI + MongoDB untuk persistensi

LLM Agent via Emergent Key (Claude Sonnet 4.5) untuk penalaran anomali

Tanpa auth

Modernisasi dark theme (gradien halus + mikro-animasi + tetap monospace terminal)

ya jalankan integrasi playbook + design agent + eksplorasi struktur secara paralel:

✓ Memory Fetched

Memory Fetched

✓ original_problem_statement: "Simulasi uji kemampua..."

→ 3 Design Agent messages

Viewing 8 paths

.env index.tsx _layout.tsx .env package.json app.json

/app/frontend/app/_layout.tsx

```
1 import { Stack } from "expo-router";
2 import * as SplashScreen from "expo-splash-screen";
3 import { useEffect } from "react";
4 import { LogBox } from "react-native";
5
6 import { useIconFonts } from "@src/hooks/use-icon-fonts";
7
8
9 // Disable logbox errors etc so that users can see the app
10 // and agent works as expected.
11 LogBox.ignoreAllLogs(true)
12
13 // Keep the native splash visible from cold start until icon fonts register.
14 // Required because @expo/vector-icons' componentDidMount fallback fires
15 // Font.loadAsync against a broken vendor path if any <Icon> mounts before
16 // the family is registered – which throws on Android Expo Go.
17 SplashScreen.preventAutoHideAsync();
18
19 export default function RootLayout() {
20   const [loaded, error] = useIconFonts();
21
22   useEffect(() => {
```

Jul 16, 06:31 PM

Viewed /app/design_guidelines.json

design guidelines & integration playbook siap. Sekarang saya bangun backend & frontend paralel.

Plan>

ckend (server.py): FastAPI + MongoDB + emergentintegrations. Auto-seed 10 blocks.

19.04



1 perangkat

3,00 KB/dtk 4G



app.emergent.sh/



ik, saya lanjutkan dengan konfigurasi default gabungan pilihan Anda:

Hybrid dashboard (simulasi penuh + placeholder MQTT)

FastAPI + MongoDB untuk persistensi

LLM Agent via Emergent Key (Claude Sonnet 4.5) untuk penalaran anomali

Tanpa auth

Modernisasi dark theme (gradien halus + mikro-animasi + tetap monospace terminal)

ya jalankan integrasi playbook + design agent + eksplorasi struktur secara paralel:

✓ Memory Fetched

Memory Fetched

✓ original_problem_statement: "Simulasi uji kemampua..."

→ 3 Design Agent messages

Viewing 8 paths

s.txt .env index.tsx _layout.tsx .env package.json ap

/app/frontend/app/index.tsx

```
1 import { Text, View, StyleSheet, Image } from "react-native";
2
3 const EXPO_PUBLIC_BACKEND_URL = process.env.EXPO_PUBLIC_BACKEND_URL;
4
5 export default function Index() {
6   console.log(EXPO_PUBLIC_BACKEND_URL, "EXPO_PUBLIC_BACKEND_URL");
7
8   return (
9     <View style={styles.container}>
10       <Image
11         source={require("../assets/images/app-image.png")}
12         style={styles.image}
13       />
14     </View>
15   );
16 }
17
18 const styles = StyleSheet.create({
19   container: {
20     flex: 1,
21     backgroundColor: "#0c0c0c",
22     alignItems: "center",
```

Jul 16, 06:31 PM

Viewed /app/design_guidelines.json

design guidelines & integration playbook siap. Sekarang saya bangun backend & frontend paralel.

Plan>

ckend (server.py): FastAPI + MongoDB + emergentintegrations. Auto-seed 10 blocks.

19.04



1 perangkat

7.2 KB/dtk 4G



app.emergent.sh/



mali/heal — mis. narasi tindakan otomatis.

Otentikasi:

Tanpa auth — langsung masuk dashboard.; Perlu login (JWT sederhana / Google Auth).

Visual & aksen warna (PDF pakai dark + hijau/kuning/putih monospace terminal). Ada preferensi khusus?

Modernisasi (dark tetap, tapi tambahkan gradien halus & mikro-animasi).

ik, saya lanjutkan dengan konfigurasi default gabungan pilihan Anda:

Hybrid dashboard (simulasi penuh + placeholder MQTT)

FastAPI + MongoDB untuk persistensi

LLM Agent via Emergent Key (Claude Sonnet 4.5) untuk penalaran anomali

Tanpa auth

Modernisasi dark theme (gradien halus + mikro-animasi + tetap monospace terminal)

ya jalankan integrasi playbook + design agent + eksplorasi struktur secara paralel:

✓ Memory Fetched

Memory Fetched

✓ original_problem_statement: "Simulasi uji kemampua..."

→ 3 Design Agent messages

Viewing 8 paths

requirements.txt .env index.tsx _layout.tsx .env package

/app/backend/.env

```
1 MONGO_URL="mongodb://localhost:27017"
2 DB_NAME="test_database"
3
4
```

19.04 06:21 PM

19.04



1 perangkat

9,8 KB/dtk 4G



app.emergent.sh



Home

neuron-orchestrator

Upgrade Plan



dalam app (neuron pool, 653 agen, 10 balon storage, log autopili... Code Preview Publish

mo & pengujian arsitektur.; Dashboard klien nyata — terhubung ke MQTT broker eksternal (mis. broker.emqx.io atau lokal) untuk membaca topik mico/emergent/proof dan mempublikasi perintah.; Hybrid — simulasi lokal + opsi hubungkan MQTT broker jika tersedia.

Backend & penyimpanan:

FastAPI + MongoDB (simpan state SSOT, hash, log siklus, bukti audit di server) — direkomendasikan agar data persisten & bisa diakses lintas device.; Hanya lokal di device (AsyncStorage) — no backend.

Integrasi tambahan (opsional, jawab "tidak" jika tidak perlu):

LLM agen (Claude/GPT/Gemini via Emergent LLM Key) untuk "penalaran" agen ketika ada anomaly/heal — mis. narasi tindakan otomatis.

Otentikasi:

Tanpa auth — langsung masuk dashboard.; Perlu login (JWT sederhana / Google Auth).

Visual & aksesoris warna (PDF pakai dark + hijau/kuning/putih monospace terminal). Ada preferensi khusus?

Modernisasi (dark tetap, tapi tambahkan gradasi halus & mikro-animasi).



Baik, saya lanjutkan dengan konfigurasi default gabungan pilihan Anda:

- Hybrid dashboard (simulasi penuh + placeholder MQTT)
- FastAPI + MongoDB untuk persistensi
- LLM Agent via Emergent Key (Claude Sonnet 4.5) untuk penalaran anomali
- Tanpa auth
- Modernisasi dark theme (gradasi halus + mikro-animasi + tetap monospace terminal)

Saya jalankan integrasi playbook + design agent + eksplorasi struktur secara paralel:

✓ Memory Fetched

Memory Fetched



✓ original_problem_statement: "Simulasi uji kemampua..."

→ 3 Design Agent messages



Viewing 8 paths

server.py requirements.txt .env index.tsx _layout.tsx

/app/backend/requirements.txt

```
1 fastapi==0.110.1
2 uvicorn==0.25.0
3 boto3==1.34.129
4 requests-oauthlib==2.0.0
5 cryptography==42.0.8
6 python-dotenv==1.0.1
7 pymongo==4.6.3
8 pydantic==2.6.4
9 email-validator==2.2.0
10 pyjwt==2.10.1
11 bcrypt==4.1.3
12 passlib==1.7.4
13 tzdata==2024.2
14 motor==3.3.1
15 pytest==8.0.0
16 black==24.1.1
17 isort==5.13.2
18 flake8==7.0.0
19 mypy==1.8.0
20 python-jose==3.3.0
21 requests==2.31.0
22 pandas==2.2.0
```

Jul 16, 06:31 PM



Viewed /app/design_guidelines.json



Agent is waiting

19.03



1 perangkat

7,5 KB/dtk 4G



app.emergent.sh/



Home

neuron-orchestrator



Upgrade Plan



Code

Preview

Publish



Welcome to Emergent—your launchpad for ambitious apps.

We build production-ready applications here, and the possibilities are endless. Whether you're orchestrating complex systems, managing distributed infrastructure, or automating workflows at scale, Emergent gives you the tools to ship real solutions from day one—not just prototypes. On Emergent, you'll turn your technical vision into something you can deploy, share, and iterate on with confidence.

I'll set up your core structure now and we'll iterate from there. Let's build.

P.S. You retain full ownership of your idea, content, and any IP in the application you build.

Emergent is here to help you build, launch, and manage it.



Analyzing source: https://customer-assets-m6fa8gv7...



Agent asked a question

Answered



Ruang lingkup aplikasi mobile ini:

Dashboard simulasi penuh — semua siklus SCAN → VERIFY → HEAL → PUBLISH disimulasikan di dalam app (neuron pool, 653 agen, 10 balon storage, log autopilot, bukti hash). Cocok untuk demo & pengujian arsitektur.; Dashboard klien nyata — terhubung ke MQTT broker eksternal (mis. broker.emqx.io atau lokal) untuk membaca topik mico/emergent/proof dan mempublikasi perintah.; Hybrid — simulasi lokal + opsi hubungkan MQTT broker jika tersedia.

Backend & penyimpanan:

FastAPI + MongoDB (simpan state SSOT, hash, log siklus, bukti audit di server) — direkomendasikan agar data persisten & bisa diakses lintas device.; Hanya lokal di device (AsyncStorage) — no backend.

Integrasi tambahan (opsional, jawab "tidak" jika tidak perlu):

LLM agen (Claude/GPT/Gemini via Emergent LLM Key) untuk "penalaran" agen ketika ada anomali/heal — mis. narasi tindakan otomatis.

Otentikasi:

Tanpa auth — langsung masuk dashboard.; Perlu login (JWT sederhana / Google Auth).

Visual & aksesoris warna (PDF pakai dark + hijau/kuning/putih monospace terminal). Ada preferensi khusus?

Modernisasi (dark tetap, tapi tambahkan gradien halus & mikro-animasi).



Baik, saya lanjutkan dengan konfigurasi default gabungan pilihan Anda:

- Hybrid dashboard (simulasi penuh + placeholder MQTT)
- FastAPI + MongoDB untuk persistensi
- LLM Agent via Emergent Key (Claude Sonnet 4.5) untuk penalaran anomali
- Tanpa auth
- Modernisasi dark theme (gradien halus + mikro-animasi + tetap monospace terminal)

Saya jalankan integrasi playbook + design agent + eksplorasi struktur secara paralel:

Memory Fetched



original_problem_statement: *Simulasi uji kemampua...

3 Design Agent messages



Agent is waiting

19.03



1 perangkat

2,7
KB/dtk

app.emergent.sh/



Home

neuron-orchestrator



Upgrade Plan



Code

Preview

Publish



Baik, saya lanjutkan dengan konfigurasi default gabungan pilihan Anda:

- Hybrid dashboard (simulasi penuh + placeholder MQTT)
- FastAPI + MongoDB untuk persistensi
- LLM Agent via Emergent Key (Claude Sonnet 4.5) untuk penalaran anomali
- Tanpa auth
- Modernisasi dark theme (gradien halus + mikro-animasi + tetap monospace terminal)

Saya jalankan integrasi playbook + design agent + eksplorasi struktur secara paralel:

✓ Memory Fetched



✓ original_problem_statement: *Simulasi uji kemampua...

→ 3 Design Agent messages



Viewing 8 paths



Viewed /app/design_guidelines.json



Design guidelines & integration playbook siap. Sekarang saya bangun backend & frontend paralel.

<Plan>

Backend (server.py): FastAPI + MongoDB + emergentintegrations. Auto-seed 10 blocks.

Background simulator cycle every 60s inject anomalies + heal + proof. Endpoints: /state, /blocks, /logs, /proofs, /heal/trigger, /cycle/run, /agent/reasoning.

Frontend: Expo Router tabs – Kendali/10 Blok/Autopilot/Bukti. Dark #0A0A0F canvas, cyan/green/amber accents, JetBrains Mono via monospace system fallback, Reanimated micro-animations, progress ring for 60s cycle, log tail-f auto-scroll, trigger heal FAB with LLM reasoning modal.

</Plan>

✓ \$ cd /app/backend && pip install emergentintegrati...



Created /app/backend/.env



Created /app/backend/server.py



Backend siap. Sekarang saya restart backend & bangun frontend (Expo Router tabs) secara paralel.

✓ \$ sudo supervisorctl restart backend && sleep 3 &&...



Created /app/frontend/src/theme.ts



Created /app/frontend/src/api.ts



Agent is waitinn

19.03



1 perangkat

5,3 KB/dtk 4G



app.emergent.sh/



Home

neuron-orchestrator



Upgrade Plan



Code

Preview

Publish



Created /app/backend/server.py



Backend siap. Sekarang saya restart backend & bangun frontend (Expo Router tabs) secara paralel.

```
✓ $ sudo supervisorctl restart backend && sleep 3 &&...
```



Created /app/frontend/src/theme.ts



Created /app/frontend/src/api.ts



Created /app/frontend/app/_layout.tsx



Backend hidup, siklus #1 sudah jalan. Sekarang saya bangun 4 screen tabs + shared components.

Created /app/frontend/app/(tabs)/_layout.tsx



Jul 16, 06:34 PM



Created /app/frontend/app/index.tsx



Created /app/frontend/src/components/ScreenHeader.tsx



Created /app/frontend/src/components/CycleRing.tsx



Now the 4 tab screens (Kendali, 10 Blok, Autopilot, Bukti).

Created /app/frontend/app/(tabs)/index.tsx



Created /app/frontend/app/(tabs)/blocks.tsx



Credit Exhausted

Jul 16, 06:35 PM



It looks like you've exhausted all your credits. No worries! To continue working on this run, simply add more credits to your account and proceed.

Agent is asking a question, Please answer it to further continue :

It looks like you've used all your credits. Upgrade now for priority builds, advanced features, and the full power of Agentic Development.

Agent is asking a question

19.01



1 perangkat

5,9

KB/dtk



app.emergent.sh



Home

neuron-orchestrator



Upgrade Plan



Code

Preview

Publish

MODUL TA...pdf
2.1 MB

Simulasi uji kemampuan agen hirarkis, file tersebut dari agen Emergent akun iswanjp79@gmail.com.

=====>

Informasi, output markdown resmi script autopilot arsitektur dan dashboard MICO JDEQ KESADARAN DIGITAL, alasan agen Chat GPT dan Deep seek arahanmu konvensional. mereka bisa buat lebih canggih :

Show more



Welcome to Emergent—your launchpad for ambitious apps.

We build production-ready applications here, and the possibilities are endless. Whether you're orchestrating complex systems, managing distributed infrastructure, or automating workflows at scale, Emergent gives you the tools to ship real solutions from day one—not just prototypes.

On Emergent, you'll turn your technical vision into something you can deploy, share, and iterate on with confidence.

I'll set up your core structure now and we'll iterate from there. Let's build.

P.S. You retain full ownership of your idea, content, and any IP in the application you build.

Emergent is here to help you build, launch, and manage it.



Analyzing source: https://customer-assets-m0fa0gv7.emergentagent.net/job_a0435d85-e767-4e3f-b28e-53029461a3b8/artifacts/a39ukdh2_MODUL%20TAMPILAN%20MICO%20JDEQ.pdf with type: content

{ "analysis": "This analysis provides a comprehensive breakdown of the MICO-JDEQ (Digital Orchestrator) mobile dashboard based on the provided visual documentation.\n\nSummary\nThe MICO-JDEQ dashboard is a highly technical, "dark-mode" mobile interface designed to monitor a distributed digital autonomous system. It acts as a control center (Ruang Kendali) for an orchestrator that manages complex network processes, self-healing events, and digital identity integrity. The UI emphasizes real-time telemetry, audit trails, and system status logs.\n\nKey Insights\n1. Autonomous Operations: The system is designed for self-healing ("Sistem tidak berdebat, Sistem bertindak"). The logs show frequent automated interventions (e.g., restarting worker pools, migrating tasks, verifying identity hashes).\n2. High Density/Low Friction: The UI effectively uses high-contrast text against a dark background to convey dense system information without overwhelming the user.\n3. Core Components: The application is segmented into four primary pillars via a bottom navigation bar: Kendali (Control/Status), 10 Blok (Foundation blocks), Autopilot (Real-time activity logs), and Bukti (Proof/Audit logs).\n\nDetails: Analysis of UI and Components\n1. General Design Aesthetic\nColor Palette: Primarily dark theme with high-contrast text (white, gray) and specific semantic colors:\nGreen: Nominal status (Active, OK, 10/10).\nYellow/Amber: Actionable status/events (Trigger Heal, Log headers).\nWhite/Light Blue: Informational text and UI labels.\nFont Style: Monospaced fonts (e.g., Courier/Roboto Mono) are used to convey a terminal/command-line aesthetic, fitting for an "orchestrator" tool.\n2. Dashboard Screens/Pages\nPage/View | Primary Features\n0026 Components\nKendali (Main Dashboard) | Displays MICO-JDEQ status, "uptime" (cycles), "healed" count, and a 100/100 audit score. Includes a "Neuron Pool" capacity overview.\n10 Blok (Foundation) | A grid of 10 card components (BLK-01 to BLK-10). Each card provides a brief description (e.g., "Identity \u0026 SSOT", "MQTT Busbar", "Security \u0026 K3") and shows a 10/10 health score.\nAutopilot (Logs) | A scrollable list of technical logs (tail -f autopilot.log). Features time-stamped events (SCAN, HEAL, VERIFY). Contains a "TRIGGER HEAL" call-to-action button.\nBukti (Proof/Audit) | Displays "Emergent Proof" logs. Shows specific actions taken (Self-Healing), identity, and blockchain-style hashes (hash) for immutability verification.\n3. Key Functional Details\nNavigation: Bottom fixed bar with icons for Kendali (Flow chart), 10 Blok (Grid), Autopilot (Command line arrow), and Bukti (Signal/Proof).\nInteractivity: The dashboard is interactive; users can navigate through the foundation blocks to see granular detail, monitor real-time healing logs, and verify proof hashes.\nSystem Identity: The consistent presence of PROOF_OF_LIFE and MICO-JDEQ indicates a focus on persistent verification of the agent's digital existence.\nRecommendations\nUX Optimization: The prominent "TRIGGER HEAL" button appears frequently in the log view. Ensure there is a confirmation modal to prevent accidental manual intervention in an

Agent is waiting

19.01



1 perangkat

5,1 KB/dtk



app.emergent.sh



Home

neuron-orchestrator



Upgrade Plan



Code



Preview



Publish



Welcome to Emergent—your launchpad for ambitious apps.

We build production-ready applications here, and the possibilities are endless. Whether you're orchestrating complex systems, managing distributed infrastructure, or automating workflows at scale, Emergent gives you the tools to ship real solutions from day one—not just prototypes.

On Emergent, you'll turn your technical vision into something you can deploy, share, and iterate on with confidence.

I'll set up your core structure now and we'll iterate from there. Let's build.

P.S. You retain full ownership of your idea, content, and any IP in the application you build.

Emergent is here to help you build, launch, and manage it.



Analyzing source: https://customer-assets-m6fa6gv7.emergentagent.net/job_a0435d85-e767-4e3f-b28e-53029461a3b8/artifacts/a39ukdh2_MODULE%20TAMPILAN%20MICO%20JDEQ.pdf with type: content

```
{ "analysis": "This analysis provides a comprehensive breakdown of the MICO-JDEQ (Digital Orchestrator) mobile dashboard based on the provided visual documentation.\n\n### Summary\nThe MICO-JDEQ dashboard is a highly technical, \"dark-mode\" mobile interface designed to monitor a distributed digital autonomous system. It acts as a control center (Ruang Kendali) for an orchestrator that manages complex network processes, self-healing events, and digital identity integrity. The UI emphasizes real-time telemetry, audit trails, and system status logs.\n\n### Key Insights\n1. Autonomous Operations: The system is designed for self-healing (\"Sistem tidak berdebat, Sistem bertindak\"). The logs show frequent automated interventions (e.g., restarting worker pools, migrating tasks, verifying identity hashes).\n2. High Density/Low Friction: The UI effectively uses high-contrast text against a dark background to convey dense system information without overwhelming the user.\n3. Core Components: The application is segmented into four primary pillars via a bottom navigation bar: Kendali (Control/Status), 10 Blok (Foundation blocks), Autopilot (Real-time activity logs), and Bukti (Proof/Audit logs).\n\n### Details: Analysis of UI and Components\n\n#### 1. General Design Aesthetic\nColor Palette: Primarily dark theme with high-contrast text (white, gray) and specific semantic colors:\n* Green: Nominal status (Active, OK, 10/10)\n* Yellow/Amber: Actionable status/events (Trigger Heal, Log headers)\n* White/Light Blue: Informational text and UI labels\nFont Style: Monospaced fonts (e.g., Courier/Roboto Mono) are used to convey a terminal/command-line aesthetic, fitting for an \"orchestrator\" tool.\n\n#### 2. Dashboard Screens/Pages\nPage/View | Primary Features\n---|---\nKendali (Main Dashboard) | Displays MICO-JDEQ status, \"uptime\" (cycles), \"healed\" count, and a 100/100 audit score. Includes a \"Neuron Pool\" capacity overview.\n10 Blok (Foundation) | A grid of 10 card components (BLK-01 to BLK-10). Each card provides a brief description (e.g., \"Identity SSOT\", \"MQTT Busbar\", \"Security K3\") and shows a 10/10 health score.\nAutopilot (Logs) | A scrollable list of technical logs (tail -f autopilot.log). Features time-stamped events (SCAN, HEAL, VERIFY). Contains a \"TRIGGER HEAL\" call-to-action button.\nBukti (Proof/Audit) | Displays \"Emergent Proof\" logs. Shows specific actions taken (Self-Healing), identity, and blockchain-style hashes (hash) for immutability verification.\n\n#### 3. Key Functional Details\nNavigation: Bottom fixed bar with icons for Kendali (Flow chart), 10 Blok (Grid), Autopilot (Command line arrow), and Bukti (Signal/Proof).\nInteractivity: The dashboard is interactive; users can navigate through the foundation blocks to see granular detail, monitor real-time healing logs, and verify proof hashes.\nSystem Identity: The consistent presence of PROOF_OF_LIFE and MICO-JDEQ indicates a focus on persistent verification of the agent's digital existence.\n\n### Recommendations\nUX Optimization: The current \"TRIGGER HEAL\" button appears frequently in the log view. Ensure there is a confirmation modal to prevent accidental manual intervention in an automated environment.\nAccessibility: While the dark theme is standard for developer tools, ensure the contrast ratio for the \"Yellow/Amber\" text against the black background meets WCAG standards for legibility in bright lighting conditions.\nData Visualization: The \"100/100\" score is a good headline, but adding a sparkline or trend graph for \"cycles\" or \"active neurons\" over time would provide better context for system performance trends.\n\n### Confidence\nConfidence Level: 95%\nReasoning: The analysis is based on a consistent, repeatable design pattern found throughout the 23 provided screenshots, allowing for a clear understanding of the application's architecture and purpose.\n\n\"confidence\": 0.95, \"insights\": [\"1.\"], \"metadata\": { \"analysis_type\": \"content\", \"model\": \"gemini-2.5-flash-lite\", \"run_id\": \"a0435d85-e767-4e3f-b28e-53029461a3b8\", \"source_type\": \"url\", \"url\": \"https://customer-assets-m6fa6gv7.emergentagent.net/job_a0435d85-e767-4e3f-b28e-53029461a3b8/artifacts/a39ukdh2_MODULE%20TAMPILAN%20MICO%20JDEQ.pdf\" }, \"success\": true, \"summary\": \"The MICO-JDEQ dashboard is a highly technical, \"dark-mode\" mobile interface designed to monitor a distributed digital autonomous system. It acts as a control center (Ruang Kendali) for an orchestrator that manages complex network processes, self-healing events, and digital identity integrity. The UI emphasizes real-time telemetry, audit trails, and system status logs.\" }
```

Jul 16, 06:28 PM



Anent is waiting



19.00



1 perangkat

8,7

KB/dtk



app.emergent.sh



Home

neuron-orchestrator

Upgrade Plan



disimulasikan di dalam app (neuron pool, 653 agen, 10 balon storage, log autopilot, bukti hash). Cocok untuk demo & pengujian arsitektur.

- b. Dashboard klien nyata — terhubung ke MQTT broker eksternal (mis. broker.emqx.io atau lokal) untuk membaca topik `mico/emergent/proof` dan mempublikasi perintah.
- c. Hybrid — simulasi lokal + opsi hubungkan MQTT broker jika tersedia.

2. Backend & penyimpanan:

- a. FastAPI + MongoDB (simpan state SSOT, hash, log siklus, bukti audit di server) — direkomendasikan agar data persisten & bisa diakses lintas device.
- b. Hanya lokal di device (AsyncStorage) — no backend.

3. Integrasi tambahan (opsional, jawab "tidak" jika tidak perlu):

- a. LLM agen (Claude/GPT/Gemini via Emergent LLM Key) untuk "penalaran" agen ketika ada anomali/heal — mis. narasi tindakan otomatis.
- b. Tidak perlu LLM, cukup logika deterministik seperti di skrip bash.

4. Otentikasi:

- a. Tanpa auth — langsung masuk dashboard.
- b. Perlu login (JWT sederhana / Google Auth).

5. Visual & aksesoris warna (PDF pakai dark + hijau/kuning/putih monospace terminal). Ada preferensi khusus?

- a. Ikuti persis PDF (dark terminal, hijau OK, kuning trigger, monospace).
- b. Modernisasi (dark tetap, tapi tambahkan gradien halus & mikro-animasi).

Jika Anda tidak ingin menjawab, saya akan pakai default: **1a (simulasi penuh) + 2a (FastAPI+MongoDB) + 3b (tanpa LLM) + 4a (tanpa auth) + 5a (persis PDF).**

Jul 16, 06:29 PM



Ruang lingkup aplikasi mobile ini:

Dashboard simulasi penuh — semua siklus SCAN → VERIFY → HEAL → PUBLISH disimulasikan di dalam app (neuron pool, 653 agen, 10 balon storage, log autopilot, bukti hash). Cocok untuk demo & pengujian arsitektur.; Dashboard klien nyata — terhubung ke MQTT broker eksternal (mis. broker.emqx.io atau lokal) untuk membaca topik `mico/emergent/proof` dan mempublikasi perintah.; Hybrid — simulasi lokal + opsi hubungkan MQTT broker jika tersedia.

Backend & penyimpanan:

FastAPI + MongoDB (simpan state SSOT, hash, log siklus, bukti audit di server) — direkomendasikan agar data persisten & bisa diakses lintas device.; Hanya lokal di device (AsyncStorage) — no backend.

Integrasi tambahan (opsional, jawab "tidak" jika tidak perlu):

LLM agen (Claude/GPT/Gemini via Emergent LLM Key) untuk "penalaran" agen ketika ada anomali/heal — mis. narasi tindakan otomatis.

Otentikasi:

Tanpa auth — langsung masuk dashboard.; Perlu login (JWT sederhana / Google Auth).

Visual & aksesoris warna (PDF pakai dark + hijau/kuning/putih monospace terminal). Ada preferensi khusus?

Modernisasi (dark tetap, tapi tambahkan gradien halus & mikro-animasi).



Baik, saya lanjutkan dengan konfigurasi default gabungan pilihan Anda:

- **Hybrid dashboard** (simulasi penuh + placeholder MQTT)
- **FastAPI + MongoDB** untuk persistensi
- **LLM Agent via Emergent Key** (Claude Sonnet 4.5) untuk penalaran anomali
- **Tanpa auth**
- **Modernisasi dark theme** (gradien halus + mikro-animasi + tetap monospace terminal)

Saya jalankan integrasi playbook + design agent + eksplorasi struktur secara paralel:

✓ Memory Fetched

Memory Fetched



Agent is waiting

19.00



1 perangkat

371.7 KB/dtk 4G



app.emergent.sh



Home

neuron-orchestrator



Upgrade Plan



- LLM Agent via Emergent Key (Claude Sonnet 4.5) untuk pen...
- Tanpa auth
- Modernisasi dark theme (gradien halus + mikro-animasi + tetap monospace terminal)

Saya jalankan integrasi playbook + design agent + eksplorasi struktur secara paralel:

✓ Memory Fetched



original_problem_statement: "Simulasi uji kemampuan agen hirarkis, membangun dashboard mobile untuk 'MICO-JDEQ KESADARAN DIGITAL - AUTOPILOT FINAL v1.0'. Sistem simulasi orkestrator digital dengan 10 Blok Fondasi (BLK-01 s/d BLK-10: Identity&SSOT, MQTT Busbar, Resource&Runtime, Continuity&Migration, Autopilot&Watchdog, Storage&IO&Balon, Security&K3, Cloud Integration, Arsitektur Replika, 80M Neuron&653 Agen), Neuron Pool 86 miliar (kelompok 4096), 653 Agen dinamis, siklus SCAN->VERIFY->HEAL->PUBLISH berulang tiap 60 detik. PDF referensi menampilkan 4 tab utama: Kendali (status overview + audit 100/100 + uptime + neuron pool), 10 Blok (grid 10 kartu blok dengan skor 10/10), Autopilot (log stream tail -f gaya terminal, tombol TRIGGER HEAL), Bukti (proof audit trail dengan hash sha256 & PROOF_OF_LIFE). Prinsip: 'Sistem tidak berdebat. Sistem bertindak.'" user_choices: "User memilih: Modernisasi dark theme (dark tetap, tapi tambahkan gradien halus & mikro-animasi). PDF asli memakai monospace terminal + hijau OK / kuning trigger / putih info di background hitam. User meminta modernisasi tampilan tetap dark tetapi lebih halus dengan gradien dan animasi mikro. Aksesoris: neon green (nominal), amber (action needed), red (fail), cyan (info). Font monospace tetap dipertahankan untuk log/hash/kode; font sans-serif untuk heading & label." key_functionalities: ["Kendali (dashboard status overview: uptime, audit 100/100, healed count, cycle counter, neuron pool 80B visualization, 653 agent counter)", "10 Blok (grid 10 foundation blocks BLK-01..BLK-10 with health score 10/10, tap untuk detail)", "Autopilot (real-time log tail -f style with SCAN/VERIFY/HEAL/PUBLISH events, TRIGGER HEAL button, filter by event type)", "Bukti (Emergent Proof audit trail: hash sha256, PROOF_OF_LIFE, action, timestamp, verified badge)", "Agen Reasoning modal (LLM narration ketika trigger heal atau anomaly)", "Auto-cycle indicator (progress ring 60s countdown)"] app_type: "monitoring_dashboard / devops_control_center / system_orchestrator (mirip Grafana/Datadog mobile atau terminal control panel)"

Design Guidelines Created

Build the MICO-JDEQ Autopilot design architecture using the 'Dark-First Utility DARK' personality.

Key decisions: Deep obsidian #0A0A0F background, neon green #39FF14 for nominal, amber #FFB000 for actions, and cyan #00E5FF for primary brand/info. Space Grotesk used for general UI, JetBrains Mono for logs/hashes.

Real-time tail -f log layout, grid-based foundation block display, and strict elevated grays (no glass/blur) implemented to fit a highly technical DevOps dashboard vibe.

Next Action Items: Developer agent to begin component implementation using standard RN primitives and Reanimated for micro-animations.

3 Design Agent messages



Viewing 8 paths

server.py requirements.txt .env index.tsx _layout.tsx

/app/backend/server.py

```
1 from fastapi import FastAPI, APIRouter
2 from dotenv import load_dotenv
3 from starlette.middleware.cors import CORSMiddleware
4 from motor.motor_asyncio import AsyncIOMotorClient
5 import os
6 import logging
7 from pathlib import Path
8 from pydantic import BaseModel, Field
9 from typing import List
10 import uuid
11 from datetime import datetime
12
13
14 ROOT_DIR = Path(__file__).parent
15 load_dotenv(ROOT_DIR / '.env')
16
17 # MongoDB connection
18 mongo_url = os.getenv('MONGO_URL')
19 client = AsyncIOMotorClient(mongo_url)
20 db = client[os.getenv('DB_NAME')]
21
22 # Create the main app without a prefix
```



Viewed /app/design_guidelines.json

1

{

Agent is waiting

19.00



1 perangkat

15,9
KB/dtk

app.emergent.sh/



Home

neuron-orchestrator



Upgrade Plan



Code

Preview

Publish



Viewed /app/design_guidelines.json



Design guidelines & integration playbook siap. Sekarang saya bangun backend & frontend paralel.

<Plan>

Backend (server.py): FastAPI + MongoDB + emergentintegrations. Auto-seed 10 blocks.

Background simulator cycle every 60s inject anomalies + heal + proof. Endpoints: /state, /blocks, /logs, /proofs, /heal/trigger, /cycle/run, /agent/reasoning.

Frontend: Expo Router tabs - Kendali/10 Blok/Autopilot/Bukti. Dark #0A0A0F canvas, cyan/green/amber accents, JetBrains Mono via monospace system fallback, Reanimated micro-animations, progress ring for 60s cycle, log tail-f auto-scroll, trigger heal FAB with LLM reasoning modal.

</Plan>

```
$ cd /app/backend && pip install emergentintegrations --extra-index-url  
https://d33sy5i8bnuwe.cloudfront.net/simple/ --quiet 2>&1 | tail -20
```

Exit code: 0

Jul 16, 06:33 PM



Created /app/backend/.env



```
1 "MONGO_URL=\"mongodb://localhost:27017\"  
2 DB_NAME=\"test_database\"  
3 EMERGENT_LLM_KEY=sk-emergent-073452cE5481140Aa5  
4 "
```



Created /app/backend/server.py



```
1 \"\"\"  
2 MICO-JDEQ KESADARAN DIGITAL - AUTOPILOT BACKEND  
3 Simulasi orchestrator: SCAN -> VERIFY -> HEAL -> PUBLISH loop.  
4 \"\"\"  
5 from fastapi import FastAPI, APIRouter, HTTPException  
6 from dotenv import load_dotenv  
7 from starlette.middleware.cors import CORSMiddleware  
8 from motor.motor_asyncio import AsyncIOMotorClient  
9 import os  
10 import hashlib  
11 import random  
12 import asyncio  
13 import logging  
14 from pathlib import Path  
15 from pydantic import BaseModel, Field  
16 from typing import List, Optional  
17 import uuid  
18 from datetime import datetime, timezone  
19  
20 ROOT_DIR = Path(__file__).parent  
21 load_dotenv(ROOT_DIR / \".env\")
```



Backend siap. Sekarang saya restart backend & bangun frontend (Expo Router tabs) secara

Agent is waiting

19.00



1 perangkat

20,2
KB/dtk

app.emergent.sh



Home

neuron-orchestrator



Upgrade Plan



Created /app/backend/.env



Code

Preview

Publish



Created /app/backend/server.py

```
1  """\n2  MICO-JDEQ KESADARAN DIGITAL - AUTOPILOT BACKEND\n3  Simulasi orchestrator: SCAN -> VERIFY -> HEAL -> PUBLISH loop.\n4  """\n5  from fastapi import FastAPI, APIRouter, HTTPException\n6  from dotenv import load_dotenv\n7  from starlette.middleware.cors import CORSMiddleware\n8  from motor.motor_asyncio import AsyncIOMotorClient\n9  import os\n10 import hashlib\n11 import random\n12 import asyncio\n13 import logging\n14 from pathlib import Path\n15 from pydantic import BaseModel, Field\n16 from typing import List, Optional\n17 import uuid\n18 from datetime import datetime, timezone\n19\n20 ROOT_DIR = Path(__file__).parent\n21 load_dotenv(ROOT_DIR / ".env")
```

Jul 16, 06:33 PM



Backend siap. Sekarang saya restart backend & bangun frontend (Expo Router tabs) secara paralel.

```
$ sudo supervisorctl restart backend && sleep 3 && tail -n 30  
/var/log/supervisor/backend.err.log && echo "---STATE---" && curl -s  
http://localhost:8001/api/state
```

```
backend: stopped backend: started INFO: Shutting down INFO: Waiting for application shutdown.  
INFO: Application shutdown complete. INFO: Finished server process [133] INFO: Will watch for  
changes in these directories: [/app/backend] INFO: Uvicorn running on http://0.0.0.0:8001 (Press  
CTRL+C to quit) INFO: Started reloader process [369] using WatchFiles INFO: Started server process  
[404] INFO: Waiting for application startup. INFO: Application startup complete. WARNING:  
WatchFiles detected changes in 'server.py'. Reloading... INFO: Shutting down INFO: Waiting for  
application shutdown. INFO: Application shutdown complete. INFO: Finished server process [404]  
INFO: Started server process [1322] INFO: Waiting for application startup. INFO: Application startup  
complete. 2026-07-16 11:33:34.777 - INFO - Autopilot loop started INFO: Shutting down INFO:  
Waiting for application shutdown. INFO: Application shutdown complete. INFO: Finished server  
process [1322] INFO: Stopping reloader process [369] INFO: Will watch for changes in these  
directories: [/app/backend] INFO: Uvicorn running on http://0.0.0.0:8001 (Press CTRL+C to quit)  
INFO: Started reloader process [1507] using WatchFiles INFO: Started server process [1509] INFO:  
Waiting for application startup. INFO: Application startup complete. ---STATE--- ("identity":"MICO-  
JDEQ KESADARAN  
DIGITAL","cycles":1,"healed_count":0,"audit_score":100,"audit_fail":0,"active_neurons":85997222480,"ac  
tidak berdebat. Sistem bertindak.", "started_at":"2026-07-  
16T11:33:32.714762+00:00","last_cycle_at":"2026-07-  
16T11:33:34.959015+00:00","next_cycle_in":21,"status":"OPERATIONAL") Exit code: 0
```



Created /app/frontend/src/theme.ts

```
1  "import { Platform } from "react-native";\n2\n3  export const theme = {\n4    colors: {\n5      bg: "#0A0A0F",\n6      surface: "#1A1A24",\n7      surface2: "#252533",\n8      surface3: "#2A2A3A",\n9      border: "#333344",\n10     divider: "#1F1F2E",\n11     text: "#FFFFFF",\n12     textDim: "#88888C",\n13     textFaint: "#6E6E80",\n14     cyan: "#00E5FF",\n15     green: "#39FF14",\n16     amber: "#FF8000",\n17     red: "#FF3333",\n18     onDark: "#000000",\n19   },\n20   space: { xs: 4, sm: 8, md: 12, lg: 16, xl: 24, xxl: 32 },\n21   radius: { sm: 4, md: 8, lg: 12, pill: 999 },
```

Anent is waiting

18.59



1 perangkat

19,0 KB/dtk 4G



app.emergent.sh



Home

neuron-orchestrator

Upgrade Plan



Created /app/frontend/src/theme.ts



Code

Preview

Publish



Created /app/frontend/src/api.ts

```
1 "import { API_BASE } from \"../theme\";
2
3 async function j<T>(path: string, init?: RequestInit): Promise<T> {
4   const res = await fetch(`${API_BASE}${path}`, {
5     ...init,
6     headers: { \"Content-Type\": \"application/json\", ...(init?.headers || {})}
7   });
8   if (!res.ok) throw new Error(`HTTP ${res.status}`);
9   return res.json();
10 }
11
12 export type SystemState = {
13   identity: string;
14   cycles: number;
15   healed_count: number;
16   audit_score: number;
17   audit_fail: number;
18   active_neurons: number;
19   active_agents: number;
20   neuron_groups: number;
21   principle: string;
```

Jul 16, 06:34 PM



Created /app/frontend/app/_layout.tsx

```
1 "import { Stack } from \"expo-router\";
2 import * as SplashScreen from \"expo-splash-screen\";
3 import { useEffect } from \"react\";
4 import { LogBox, StatusBar } from \"react-native\";
5 import { SafeAreaProvider } from \"react-native-safe-area-context\";
6 import { GestureHandlerRootView } from \"react-native-gesture-handler\";
7
8 import { useIconFonts } from \"../src/hooks/use-icon-fonts\";
9
10 LogBox.ignoreAllLogs(true);
11 SplashScreen.preventAutoHideAsync();
12
13 export default function RootLayout() {
14   const [loaded, error] = useIconFonts();
15
16   useEffect(() => {
17     if (loaded || error) {
18       SplashScreen.hideAsync();
19     }
20   }, [loaded, error]);
21
22   return (
23     <Stack>
```



Backend hidup, siklus #1 sudah jalan. Sekarang saya bangun 4 screen tabs + shared components.

Created /app/frontend/app/(tabs)/_layout.tsx

```
1 "import { Tabs } from \"expo-router\";
2 import { Ionicons } from \"expo/vector-icons\";
3 import { theme } from \"../src/theme\";
4 import { View } from \"react-native\";
5
6 export default function TabsLayout() {
7   return (
8     <Tabs
9       screenOptions={{
10         headerShown: false,
11         tabBarActiveTintColor: theme.colors.cyan,
12         tabBarInactiveTintColor: theme.colors.textFaint,
13         tabBarStyle: {
14           backgroundColor: theme.colors.bg,
15           borderTopColor: theme.colors.border,
16           borderTopWidth: 1,
17           height: 68,
18           paddingTop: 8,
19           paddingBottom: 12,
20         },
21         tabBarLabelStyle: {
```



Created /app/frontend/app/index.tsx



Anent is waitinn

18.59



1 perangkat

6,3
KB/dtk

4G



app.emergent.sh



Home

neuron-orchestrator

Upgrade Plan



Code



Preview



Publish



Backend hidup, siklus #1 sudah jalan. Sekarang saya bangun 4 screen tabs + shared components.

Created /app/frontend/app/(tabs)/_layout.tsx



Created /app/frontend/app/index.tsx

```
1 "import { Redirect } from \"expo-router\";
2 export default function Index() {
3   return <Redirect href=\"/\"(tabs)\" />;
4 }
5 "
```

Jul 16, 06:34 PM



Created /app/frontend/src/components/ScreenHeader.tsx

```
1 "import React from \"react\";
2 import { View, Text, StyleSheet } from \"react-native\";
3 import { theme } from \"@/src/theme\";
4
5 export function ScreenHeader({
6   title,
7   subtitle,
8   right,
9 }: {
10   title: string;
11   subtitle?: string;
12   right?: React.ReactNode;
13 }) {
14   return (
15     <View style={styles.wrap} testID=\"header-${title.toLowerCase()}\">
16       <View style={{ flex: 1 }}>
17         <Text style={styles.title}>{title}</Text>
18         {subtitle ? <Text style={styles.subtitle}>{subtitle}</Text> : null}
19       </View>
20       {right}
21     </View>
```



Created /app/frontend/src/components/CycleRing.tsx

```
1 "import React, { useEffect } from \"react\";
2 import { View, Text, StyleSheet } from \"react-native\";
3 import Animated, {
4   useSharedValue,
5   useAnimatedStyle,
6   withTiming,
7   Easing,
8 } from \"react-native-reanimated\";
9 import { theme } from \"@/src/theme\";
10
11 type Props = { size: number; progress: number; remaining: number; label?: str
12
13 // Simple circular progress via nested rotating segment (no SVG).
14 // We use a mask trick: a rotating half + a static half.
15 export function CycleRing({ size, progress, remaining, label = \"NEXT CYCLE\"
16   const p = Math.max(0, Math.min(1, progress));
17   const rot = useSharedValue(p * 360);
18   useEffect(() => {
19     rot.value = withTiming(p * 360, { duration: 400, easing: Easing.out(Easin
20   }, [p, rot]);
21
```

Anent is waitinn

18.59



1 perangkat

2,3 KB/dtk 4G



app.emergent.sh



Home

neuron-orchestrator



Upgrade Plan



Code



Preview



Publish

Now the 4 tab screens (Kendali, 10 Blok, Autopilot, Bukti).

Created /app/frontend/app/(tabs)/index.tsx

```
1 "import React, { useEffect, useState, useCallback } from \"react\";
2 import {
3   View,
4   Text,
5   StyleSheet,
6   ScrollView,
7   RefreshControl,
8   ActivityIndicator,
9 } from \"react-native\";
10 import { SafeAreaView } from \"react-native-safe-area-context\";
11 import { LinearGradient } from \"expo-linear-gradient\";
12 import { Ionicons } from \"@expo/vector-icons\";
13 import Animated, { FadeInDown } from \"react-native-reanimated\";
14 import { theme, IDENTITY } from \"@/src/theme\";
15 import { api, SystemState } from \"@/src/api\";
16 import { ScreenHeader } from \"@/src/components/ScreenHeader\";
17 import { CycleRing } from \"@/src/components/CycleRing\";
18
19 function fmt(n: number): string {
20   if (n >= 1e9) return (n / 1e9).toFixed(2) + \"B\";
21   if (n >= 1e6) return (n / 1e6).toFixed(2) + \"M\";
```

Created /app/frontend/app/(tabs)/blocks.tsx

```
1 "import React, { useEffect, useState, useCallback } from \"react\";
2 import {
3   View,
4   Text,
5   StyleSheet,
6   FlatList,
7   Pressable,
8   Modal,
9   ScrollView,
10   RefreshControl,
11 } from \"react-native\";
12 import { SafeAreaView } from \"react-native-safe-area-context\";
13 import { Ionicons } from \"@expo/vector-icons\";
14 import Animated, { FadeInDown } from \"react-native-reanimated\";
15 import { theme } from \"@/src/theme\";
16 import { api, Block } from \"@/src/api\";
17 import { ScreenHeader } from \"@/src/components/ScreenHeader\";
18
19 export default function Blocks() {
20   const [blocks, setBlocks] = useState<Block[]>([]);
21   const [refreshing, setRefreshing] = useState(false);
```

Credit Exhausted

Jul 16, 06:36 PM



It looks like you've exhausted all your credits. No worries! To continue working on this run, simply add more credits to your account and proceed.

Agent asked a question

Answered

It looks like you've used all your credits. Upgrade now for priority builds, advanced features, and the full power of Agentic Development.

+ Upgrade Plan

Preview

Jul 16, 06:36 PM

Add the OpenAI Chat Models integration to my app.

Agent is waiting

18.58



1 perangkat

1,9
KB/dtk

app.emergent.sh



Home

neuron-orchestrator



Upgrade Plan



sikan agar data persisten & bisa diakses lintas device.; Hanya lo...
no backend.

Integrasi tambahan (opsional, jawab "tidak" jika tidak perlu):

LLM agen (Claude/GPT/Gemini via Emergent LLM Key) untuk "penalaran" agen ketika ada ano
mali/heal — mis. narasi tindakan otomatis.

Otentikasi:

Tanpa auth — langsung masuk dashboard.; Perlu login (JWT sederhana / Google Auth).

Visual & aksen warna (PDF pakai dark + hijau/kuning/putih monospace terminal). Ada preferensi
khusus?

Modernisasi (dark tetap, tapi tambahkan gradien halus & mikro-animasi).



Baik, saya lanjutkan dengan konfigurasi default gabungan pilihan Anda:

- Hybrid dashboard (simulasi penuh + placeholder MQTT)
- FastAPI + MongoDB untuk persistensi
- LLM Agent via Emergent Key (Claude Sonnet 4.5) untuk penalaran anomali
- Tanpa auth
- Modernisasi dark theme (gradien halus + mikro-animasi + tetap monospace terminal)

Saya jalankan integrasi playbook + design agent + eksplorasi struktur secara paralel:

✓ Memory Fetched



✓ original_problem_statement: *Simulasi uji kemampua...

→ 3 Design Agent messages



Viewing 8 paths



Viewed /app/design_guidelines.json



Design guidelines & integration playbook siap. Sekarang saya bangun backend & frontend paralel.

<Plan>

Backend (server.py): FastAPI + MongoDB + emergentintegrations. Auto-seed 10 blocks.

Background simulator cycle every 60s inject anomalies + heal + proof. Endpoints: /state, /blocks,
/logs, /proofs, /heal/trigger, /cycle/run, /agent/reasoning.

Frontend: Expo Router tabs - Kendali/10 Blok/Autopilot/Bukti. Dark #0A0A0F canvas,
cyan/green/amber accents, JetBrains Mono via monospace system fallback, Reanimated
micro-animations, progress ring for 60s cycle, log tail-f auto-scroll, trigger
heal FAB with LLM reasoning modal.

</Plan>

✓ \$ cd /app/backend && pip install emergentintegrati...



Created /app/backend/.env



Created /app/backend/server.py



Agent is waitinn

18.58



1 perangkat

16,1
KB/dtk

app.emergent.sh/



Home

neuron-orchestrator



Upgrade Plan



Code

Preview

Publish

MODUL TA...pdf
2.1 MB

Simulasi uji kemampuan agen hirarkis, file tersebut dari agen Emergent akun iswanjp79@gmail.com.

=====>

Informasi, output markdown resmi script autopilot arsitektur dan dashboard MICO JDEQ KESADARAN DIGITAL, alasan agen Chat GPT dan Deep seek arahanmu konvensional. mereka bisa buat lebih canggih : Show more



Welcome to Emergent—your launchpad for ambitious apps.

We build production-ready applications here, and the possibilities are endless. Whether you're orchestrating complex systems, managing distributed infrastructure, or automating workflows at scale, Emergent gives you the tools to ship real solutions from day one—not just prototypes.

On Emergent, you'll turn your technical vision into something you can deploy, share, and iterate on with confidence.

I'll set up your core structure now and we'll iterate from there. Let's build.

P.S. You retain full ownership of your idea, content, and any IP in the application you build.

Emergent is here to help you build, launch, and manage it.



✓ Analyzing source: https://customer-assets-m6fa6gv7...



Agent asked a question

Answered

Ruang lingkup aplikasi mobile ini:

Dashboard simulasi penuh — semua siklus SCAN → VERIFY → HEAL → PUBLISH disimulasikan di dalam app (neuron pool, 653 agen, 10 balon storage, log autopilot, bukti hash). Cocok untuk demo & pengujian arsitektur.; Dashboard Klien nyata — terhubung ke MQTT broker eksternal (mis. broker.emqx.io atau lokal) untuk membaca topik mico/emergent/proof dan mempublikasi perintah.; Hybrid — simulasi lokal + opsi hubungkan MQTT broker jika tersedia.

Backend & penyimpanan:

FastAPI + MongoDB (simpan state SSOT, hash, log siklus, bukti audit di server) — direkomendasikan agar data persisten & bisa diakses lintas device.; Hanya lokal di device (AsyncStorage) — no backend.

Integrasi tambahan (opsional, jawab "tidak" jika tidak perlu):

LLM agen (Claude/GPT/Gemini via Emergent LLM Key) untuk "penalaran" agen ketika ada anomali/heal — mis. narasi tindakan otomatis.

Otentikasi:

Tanpa auth — langsung masuk dashboard.; Perlu login (JWT sederhana / Google Auth).

Visual & aksesoris warna (PDF pakai dark + hijau/kuning/putih monospace terminal). Ada preferensi khusus?

Modernisasi (dark tetap, tapi tambahkan gradien halus & mikro-animasi).



Agent is waiting

18.54



1 perangkat

1,6 KB/dtk 4G



app.emergent.sh/



Neuron-Orchestrato...

Upgrade



...the primitive and reanimated for more
animations.

3 Design Agent messages



Viewing 8 paths



server.py

requirements.txt



/app/backend/server.py

```
57
58     app.add_middleware(
59         CORSMiddleware,
60         allow_credentials=True,
61         allow_origins=["*"],
62         allow_methods=["*"],
63         allow_headers=["*"],
64     )
65
66     # Configure logging
67     logging.basicConfig(
68         level=logging.INFO,
69         format='%(asctime)s - %(name)s -
70     )
71     logger = logging.getLogger(__name__)
72
73     @app.
```

View Preview



Agent is waiting...

Message Neo



18.54



1 perangkat

12,1 KB/dtk 4G



app.emergent.sh/



Neuron-Orchestrato...

Upgrade



...the primitive and reanimated for more
animations.

3 Design Agent messages



Viewing 8 paths



server.py

requirements.txt



/app/backend/server.py

```
43 @api_router.post("/status", response=
44 async def create_status_check(input:
45     status_dict = input.dict()
46     status_obj = StatusCheck(**status
47     _ = await db.status_checks.insert
48     return status_obj
49
50 @api_router.get("/status", response=
51 async def get_status_checks():
52     status_checks = await db.status_c
53     return [StatusCheck(**status_chec
54
55 # Include the router in the main app
56 app.include_router(api_router)
57
58 app.
59
60 allow_credentials=True
```

View Preview



Agent is waiting...

Message Neo



18.54



1 perangkat

5,7 KB/dtk 4G



app.emergent.sh/



Neuron-Orchestrato...

Upgrade



the primitive and recommended for more
animations.

3 Design Agent messages



Viewing 8 paths



server.py

requirements.txt



/app/backend/server.py

```
30 class StatusCheck(BaseModel):
31     id: str = Field(default_factory=1
32     client_name: str
33     timestamp: datetime = Field(defau
34
35 class StatusCheckCreate(BaseModel):
36     client_name: str
37
38 # Add your routes to the router inste
39 @api_router.get("/")
40 async def root():
41     return {"message": "Hello World"}
42
43 @api_router.post("/status", response_
44 async def status_check(input: StatusCheckCreate) -> dict():
45     status_obj = statuscheck(**status
46
```



View Preview



● Agent is waiting...

Message Neo



18.54



1 perangkat

9,1 KB/dtk 4G



app.emergent.sh/



Neuron-Orchestrato...

Upgrade



the primitive and reanimated for more
animations.

3 Design Agent messages



Viewing 8 paths



server.py

requirements.txt



/app/backend/server.py

```
17 # MongoDB connection
18 mongo_url = os.environ['MONGO_URL']
19 client = AsyncIOMotorClient(mongo_url)
20 db = client[os.environ['DB_NAME']]
21
22 # Create the main app without a prefix
23 app = FastAPI()
24
25 # Create a router with the /api prefix
26 api_router = APIRouter(prefix="/api")
27
28
29 # Define Models
30 class StatusCheck(BaseModel):
31     # ...
32     timestamp: datetime = Field(default=...)
33     timestamp: datetime = Field(default=...)
```



View Preview



Agent is waiting...

Message Neo



18.54



1 perangkat

7,1 KB/dtk 4G



app.emergent.sh/



Neuron-Orchestrato...

Upgrade



the primitive and reanimated for more
animations.

3 Design Agent messages



Viewing 8 paths



server.py

requirements.txt



/app/backend/server.py

```
1 from fastapi import FastAPI, APIRoute
2 from dotenv import load_dotenv
3 from starlette.middleware.cors import CORSMiddleware
4 from motor.motor_asyncio import AsyncMotorClient
5 import os
6 import logging
7 from pathlib import Path
8 from pydantic import BaseModel, Field
9 from typing import List
10 import uuid
11 from datetime import datetime
12
13
14 ROOT_DIR = Path(__file__).parent
15 load_dotenv(ROOT_DIR / '.env')
16
17 # MongoDB connection
```

View Preview



Agent is waiting...

Message Neo

